National University of Computer & Emerging Sciences, Lahore

AI2002 – Artificial Intelligence (BS-SE-5A) — Project Phase 3: Model Implementation & Initial Results

Lightweight Speech Emotion Recognition Using Hand-Crafted Acoustic Features: Cross-Dataset Generalization and Gender-Bias Analysis

Group ID	F26-11	**Track**	B – Research & Development
Members	22L-7786 (Lead), 23L-3002, 23L-3102	**Instructor**	Hajra Waheed
Code	F26-11.ipynb (also F26-11.py) · https://github.com/junaidshahid-dev/Ai-project (folder phase3/)		

1. Model Selection and Justification

The task is 7-class speech emotion recognition (neutral, happy, sad, angry, fearful, disgust, surprised) from the hand-crafted features prepared in Phase 2. Because this is a Track B project that re-examines a published method (Chowdhury et al., *Sci. Rep.* 15, 11824, 2025, which reports 97.57% on RAVDESS), our main models are the **paper's own lightweight deep models with its hyper-parameters**. Two classical baselines tell us whether the deep models add anything beyond simple feature statistics.

Model	Input	Why we chose it
SVM, RBF kernel (C = 10)	68 values: mean + std of the 34 features over time	Standard strong baseline for small datasets; trains in about 20 seconds.
Random Forest (300 trees)	same 68 values	Non-linear, robust, needs almost no tuning.
1D-CNN (paper; 616,615 parameters)	3,672-value feature vector	Convolutions learn local patterns along the feature vector; small enough to train on a CPU.
CNN-BiLSTM (paper; 688,807 parameters)	3,672-value feature vector	A bidirectional LSTM reads the convolved sequence in both directions to capture longer-range order.
Ensemble (paper's final model)	probabilities of the two deep models	Average of the two outputs; combining different models reduces their individual errors.

Larger pre-trained speech models (e.g. wav2vec 2.0) would probably score higher, but our research question is whether *this* lightweight model and its reported accuracy hold up on unseen speakers and an unseen dataset, so we keep the paper's architecture (its Table 2) and training settings (its Table 3) unchanged.

2. Implementation Details

- **Python 3.13** in a Jupyter notebook; all training on a 2-core laptop CPU (Intel i7-6600U, no GPU). Every random seed is 42.
- **TensorFlow 2.21 / Keras 3**: the deep models (Conv1D, BatchNormalization, MaxPooling1D, Dropout, Bidirectional LSTM, Dense), the Adam optimiser and the EarlyStopping / ReduceLROnPlateau callbacks.
- **scikit-learn 1.6**: SVM, Random Forest, StandardScaler, stratified train_test_split and the metrics (accuracy, macro-F1, classification report, confusion matrix).
- **librosa 0.11**: audio loading and features for new .wav files; **NumPy / pandas** for arrays and tables; **Matplotlib** for figures.

Code structure (one small function per step, all in `F26-11.ipynb`):

Function	What it does
`load_features`	loads the Phase 2 feature cache and the label table
`random_split`	makes the stratified 80/10/10 train / validation / test clip indices
`get_frames`, `to_vectors`, `to_summary`	select clips (with augmented copies for training) and build the 3,672-value or 68-value inputs
`conv_block`, `build_model`	build the paper's 1D-CNN or CNN-BiLSTM
`train_model`, `train_all`	fit a deep model; train all four models on one split
`predict_all`, `evaluate`	class probabilities (and ensemble); accuracy and macro-F1
`extract_features`, `predict_emotion`	raw .wav file → features → predicted emotion

3. Code Workflow / Pipeline

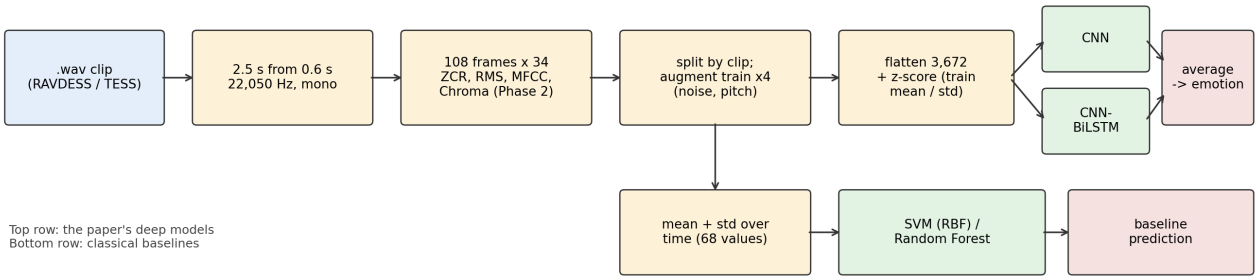


Figure 1. Pipeline from a .wav file to a predicted emotion.

1. **Input:** the Phase 2 cache `features/ravdess.npz` (1,440 clips × 4 versions × 108 frames × 34 features) and `features/tess.npz` (2,800 clips), with labels, actor and gender.
2. **Split RAVDESS by clip** (80/10/10). Only training clips use their 3 augmented copies (noise, pitch shift, both), so no version of a test clip is seen in training. TESS is never trained on.
3. **Model input:** deep models get the flattened 3,672-value vector, z-scored with the training mean / std; the baselines get the 68-value summary.
4. **Train** the four models (`train_all`), then **predict** class probabilities; the ensemble is the average of the CNN and CNN-BiLSTM probabilities.
5. **Output:** the emotion with the highest probability and its confidence; on a test set, accuracy, macro-F1 and a confusion matrix. `predict_emotion(path)` runs the whole chain on a raw file.

4. Training Process

Data split (stratified by emotion, seed 42; validation is only used for early stopping and the learning-rate schedule):

Experiment	Train clips (samples)	Validation	Test	Question
1. RAVDESS 80/10/10 (paper)	1,152 (4,608)	144	144 RAVDESS	Does the model work?
2. TESS (Exp. 1 models)	—	—	2,800 TESS	Works on a new corpus?

Setting (paper Table 3)	Value	Training run (Exp. 1)	CNN	CNN-BiLSTM
Optimiser / loss	Adam, lr 0.001 / cross-entropy	Parameters	616,615	688,807
Batch / max. epochs	64 / 100	Epochs until early stop	18	21
Early stopping	patience 5 on val. accuracy	Best validation accuracy	75.7%	79.9%
Learning rate	halved after 3 flat epochs	Training time (CPU)	72 min	95 min
Regularisation	dropout 0.2, batch norm.	SVM / RF time	20 s	11 s

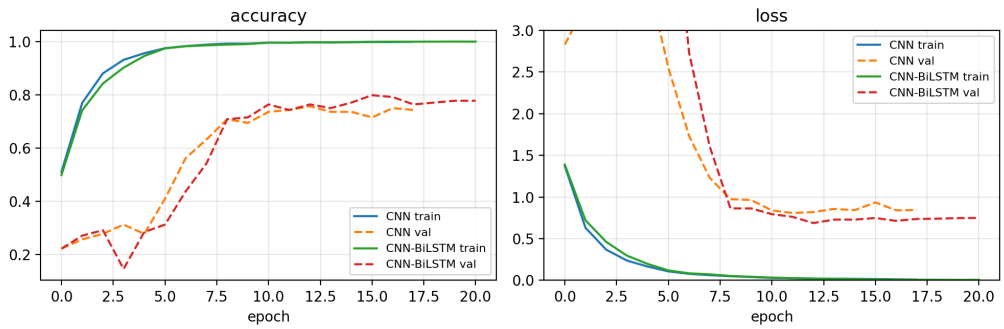


Figure 2. Experiment 1 training curves (solid = training, dashed = validation).

5. Initial Results

Model	Exp. 1: RAVDESS test (144 clips)	Exp. 2: TESS, unseen dataset (2,800 clips)
SVM	77.1 / 76.7	22.4 / 15.6
Random Forest	67.4 / 66.0	30.8 / 25.3
CNN	76.4 / 76.5	23.0 / 20.8
CNN-BiLSTM	77.1 / 75.9	17.1 / 18.5
Ensemble	77.1 / 75.9	21.0 / 19.9

Table: test accuracy / macro-F1 (%). Chance level is 20.0% on RAVDESS (always predicting *neutral*) and 14.3% on TESS.

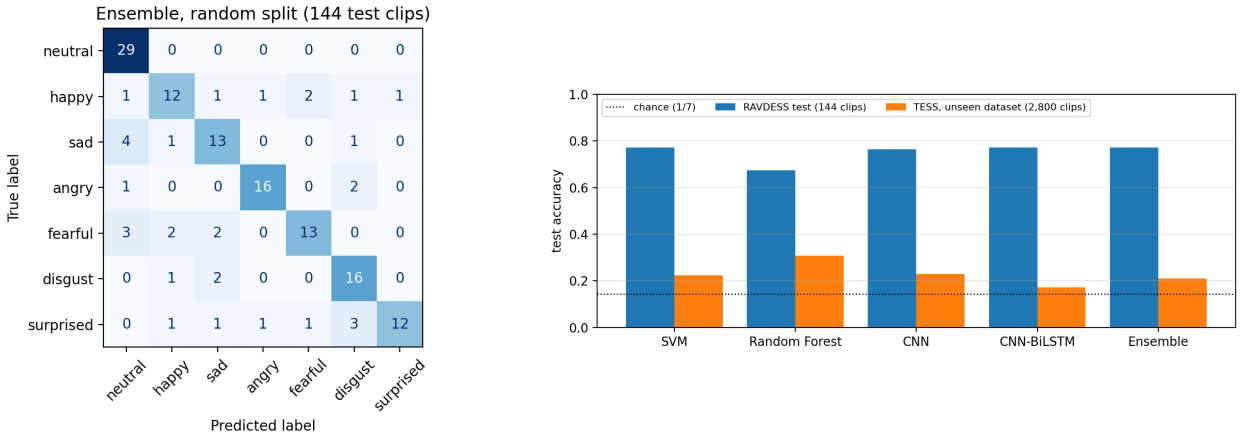


Figure 3. Left: ensemble confusion matrix on the RAVDESS test set. Right: accuracy of every model on RAVDESS and on the unseen TESS dataset.

Working model, end to end. `predict_emotion` on six raw RAVDESS test files (true → predicted, confidence): neutral → neutral (79%); fearful → fearful (≈100%); fearful → sad (≈100%); angry → angry (82%); neutral → neutral (58%); neutral → neutral (42%). Five of six are correct, and the wrong one is predicted with ≈100% confidence, so the model can be confidently wrong. The features it computes from a raw file are identical to the Phase 2 cache (maximum difference 0.0).

- **The model works on RAVDESS.** The ensemble classifies 111 of the 144 test clips correctly (77.1%, macro-F1 75.9%), almost four times the 20% chance level. In this run the deep models are level with the SVM (CNN 76.4%, CNN-BiLSTM and ensemble 77.1%, SVM 77.1%), and the Random Forest is weakest (67.4%). With 144 test clips one clip is worth 0.7 points, so gaps of a few points between models are within run-to-run variation (see *Reproducibility* in Section 6).
- **Training behaviour (Fig. 2).** Training accuracy reaches 99% by epoch 10 while validation accuracy levels off at about 70–80%, so both models overfit the 4,608 training samples. The learning rate was halved once for the CNN and twice for the CNN-BiLSTM; early stopping kept the best epoch (CNN 75.7% at epoch 13, CNN-BiLSTM 79.9% at epoch 16) and ended training 5 epochs later.
- **Per emotion (Fig. 3, left).** Neutral is always recognised (29 of 29); angry and disgust come next (16 of 19 each), while happy and surprised are hardest (12 of 19). Most errors go towards neutral: 4 sad and 3 fearful clips are predicted as neutral, so quieter emotions are easily mistaken for neutral speech.
- **Unseen dataset (TESS).** All models fall to 17–31% accuracy, close to the 14.3% chance level (ensemble 21.0%). The models learned cues specific to RAVDESS (its speakers, recording setup and clip length: 41.5% of TESS feature frames are zero padding against 0.02% for RAVDESS) rather than general emotion cues. This generalisation gap is the central question of our project.

Further experiments. The same model code is also run under stricter protocols by `scripts/run_experiments.py` (about 20 more training runs, so not repeated in this notebook). With **unseen speakers** (4-fold, 6 held-out actors per fold) the ensemble reaches 58.5% ± 1.9, and it is clearly better on female than on male voices (66.2% vs 50.7%, $p < 0.001$). If augmentation is done *before* the split, accuracy jumps to 96.5%, close to the paper's 97.57%, which shows how the paper's number can be inflated by leakage.

6. Challenges Faced

- **Details missing from the paper.** It does not say whether augmentation happens before or after the split, how the ensemble is trained, or where RAVDESS *calm* goes. We split by clip first (no leakage), train the two members separately and average them, and merge calm into neutral (this matches the 288 neutral clips in the paper's Table 1).
- **Inconsistent parameter table.** The paper lists 6,176 parameters for the last convolution, which does not fit the 128-channel Bi-LSTM output before it. We followed the layer description, so our CNN-BiLSTM has 688,807 parameters instead of 672,423.
- **Training time on a CPU.** Convolving a 3,672-step input with 128 filters is slow without a GPU: on our 2-core laptop (Intel i7-6600U) one epoch takes about 3.7 minutes, so training the two deep models took 2.8 hours. We therefore train once in the notebook and run the many-fold experiments separately with the experiment script.
- **Avoiding inflated results.** Augmenting before splitting would put copies of test clips into training, and normalising with all data would leak test statistics. Both are prevented in the code. *Neutral* is 20% of RAVDESS, so we report macro-F1 next to accuracy.
- **Reproducibility.** All seeds are fixed, but the hardware and library versions still change the trained models: an earlier run of the same model code on a faster machine with newer NumPy / scikit-learn gave the ensemble 81.9% and the Random Forest 69.4%, against 77.1% and 67.4% here (the SVM gave 77.1% both times). All numbers in this report come from one documented run of the notebook.

To reproduce: run Phase 2 (data and feature cache), then `phase3/F26-11.ipynb` top to bottom (about 3 hours on our 2-core laptop CPU). Trained models are saved in `phase3/models/`; `phase3/HOW_IT_WORKS.md` explains every function.

References. J. H. Chowdhury, S. Ramanna, K. Kotecha, “Speech emotion recognition with light weight deep neural ensemble model using hand crafted features,” *Sci. Rep.* 15, 11824 (2025). S. R. Livingstone, F. A. Russo, “RAVDESS,” *PLoS ONE* 13(5), 2018. M. K. Pichora-Fuller, K. Dupuis, “TESS,” Borealis, 2020. TensorFlow / Keras, scikit-learn, librosa documentation.